

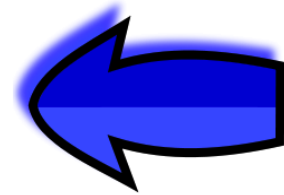


Chapter 11: The Memory System

Basic Computer Architecture

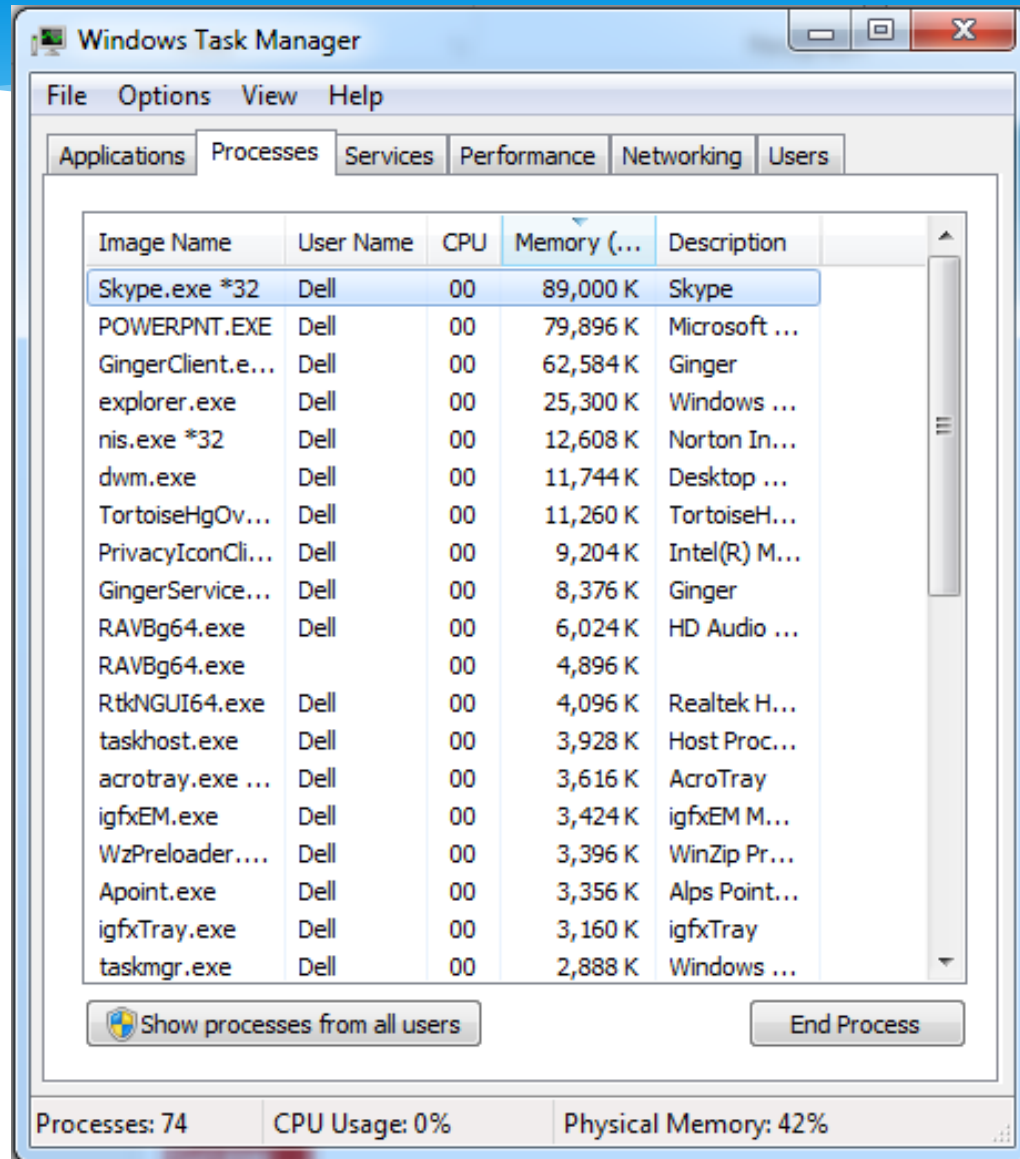
Outline

- * Overview of the Memory System
- * Caches
- * Details of the Memory System
- * Virtual Memory



Need for a Fast Memory System

- * We have up till now **assumed** that the **memory** is one large array of bytes
 - * Starts at 0, and ends at $(2^{32} - 1)$
 - * Takes **1 cycle** to access memory (read/write)
- * All programs share the memory
 - * We somehow **magically** avoid overlaps between programs running on the same processor
 - * All our **programs** require less than 4 GB of space



The screenshot shows the Windows Task Manager window with the 'Processes' tab selected. The window title is 'Windows Task Manager'. The menu bar includes 'File', 'Options', 'View', and 'Help'. The tabs at the top are 'Applications', 'Processes' (active), 'Services', 'Performance', 'Networking', and 'Users'. The main area displays a table of running processes. The table has columns: 'Image Name', 'User Name', 'CPU', 'Memory (...)', and 'Description'. The processes are listed in descending order of memory usage. At the bottom of the window, there are three status boxes: 'Processes: 74', 'CPU Usage: 0%', and 'Physical Memory: 42%'. There are also buttons for 'Show processes from all users' and 'End Process'.

Image Name	User Name	CPU	Memory (...)	Description
Skype.exe *32	Dell	00	89,000 K	Skype
POWERPNT.EXE	Dell	00	79,896 K	Microsoft ...
GingerClient.e...	Dell	00	62,584 K	Ginger
explorer.exe	Dell	00	25,300 K	Windows ...
nis.exe *32	Dell	00	12,608 K	Norton In...
dwm.exe	Dell	00	11,744 K	Desktop ...
TortoiseHgOv...	Dell	00	11,260 K	TortoiseH...
PrivacyIconCli...	Dell	00	9,204 K	Intel(R) M...
GingerService...	Dell	00	8,376 K	Ginger
RAVBg64.exe	Dell	00	6,024 K	HD Audio ...
RAVBg64.exe		00	4,896 K	
RtkNGUI64.exe	Dell	00	4,096 K	Realtek H...
taskhost.exe	Dell	00	3,928 K	Host Proc...
acrotray.exe ...	Dell	00	3,616 K	AcroTray
igfxEM.exe	Dell	00	3,424 K	igfxEM M...
WzPreloader....	Dell	00	3,396 K	WinZip Pr...
Apoint.exe	Dell	00	3,356 K	Alps Point...
igfxTray.exe	Dell	00	3,160 K	igfxTray
taskmgr.exe	Dell	00	2,888 K	Windows ...

Processes: 74 CPU Usage: 0% Physical Memory: 42%

All the programs running on my machine. The CPU of course runs one program at a time. Switches between programs periodically.

Regarding all the memory being homogeneous → NOT TRUE

Cell Type	Area	Typical Latency
Master Slave D flip flop	$0.8 \mu m^2$	Fraction of a cycle
SRAM cell in an array	$0.08 \mu m^2$	1-5 cycles
DRAM cell in an array	$0.005 \mu m^2$	50-200 cycles

Typical Values

- * Should we make our memory using only flip-flops ?
 - * **10X** the area of a memory with SRAM cells
 - * **160X** the area of a memory with DRAM cells
 - * Significantly more power !!!

Tradeoffs

- * Tradeoffs

- * Area, Power, and Latency

- * Increase Area → Reduce latency, increase power

- * Reduce latency → increase area, increase power

- * Reduce power → reduce area, increase latency

- * We cannot have
the best of all worlds



What do we do ?

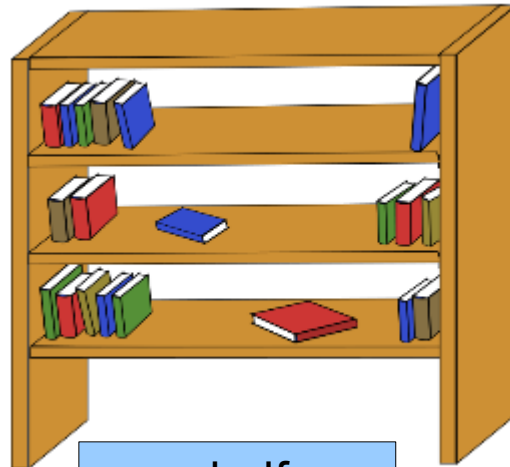
- * We cannot create a memory of just flip flops
 - * We will hardly be able to store anything
- * We cannot create a memory of just SRAM cells
 - * We need more storage, and we will not have a 1 cycle latency
- * We cannot create a memory of DRAM cells
 - * We cannot afford 50+ cycles per access

Memory Access Latency

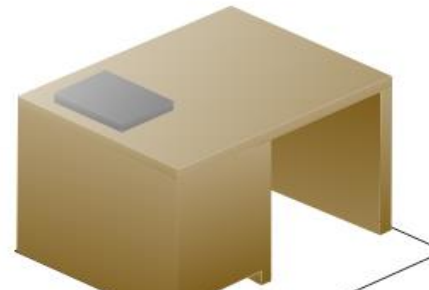
- * What does memory access latency depend on ?
 - * Size of the memory → larger is the size, slower it is
 - * Number of ports → More are the ports (parallel accesses/cycle), slower is the memory
 - * Technology used → SRAM, DRAM, flip-flops

Solution : Leverage Patterns

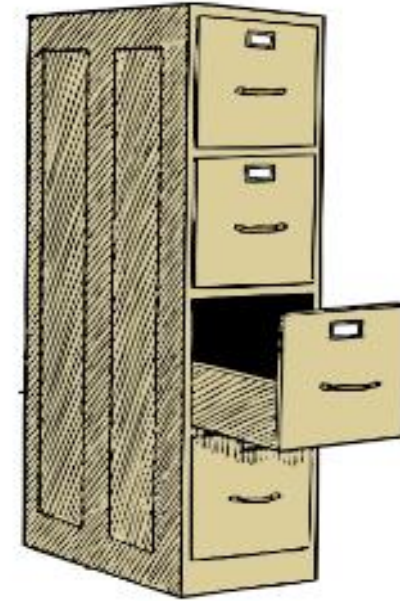
- * Look at an example in real life
 - * Sofia's workplace



shelf



desk



cabinet

A Protocol with Books

- * Sofia keeps the
 - * **most frequently** accessed books on her desk
 - * **slightly less frequently** accessed books on the shelf
 - * **rarely accessed** books in the cabinet
- * Why ?
 - * She tends to read the same set of books over and over again, in the same window of time → **Temporal Locality**

Protocol – II

- * If Sofia takes a **computer architecture** course
 - * She has comp. architecture books on her **desk**
- * After the course is over
 - * The **architecture** books go back to the **shelf**
 - * And, **vacation planning** books come to the desk
 - * **Idea** : Bring all the **vacation planning** books in one go. If she requires one, in high likelihood she might require similar books in the near future.



Temporal and Spatial Locality

Temporal Locality

It is a concept that states that if a resource is accessed at some point of time, then most likely it will be accessed again in a short period of time.

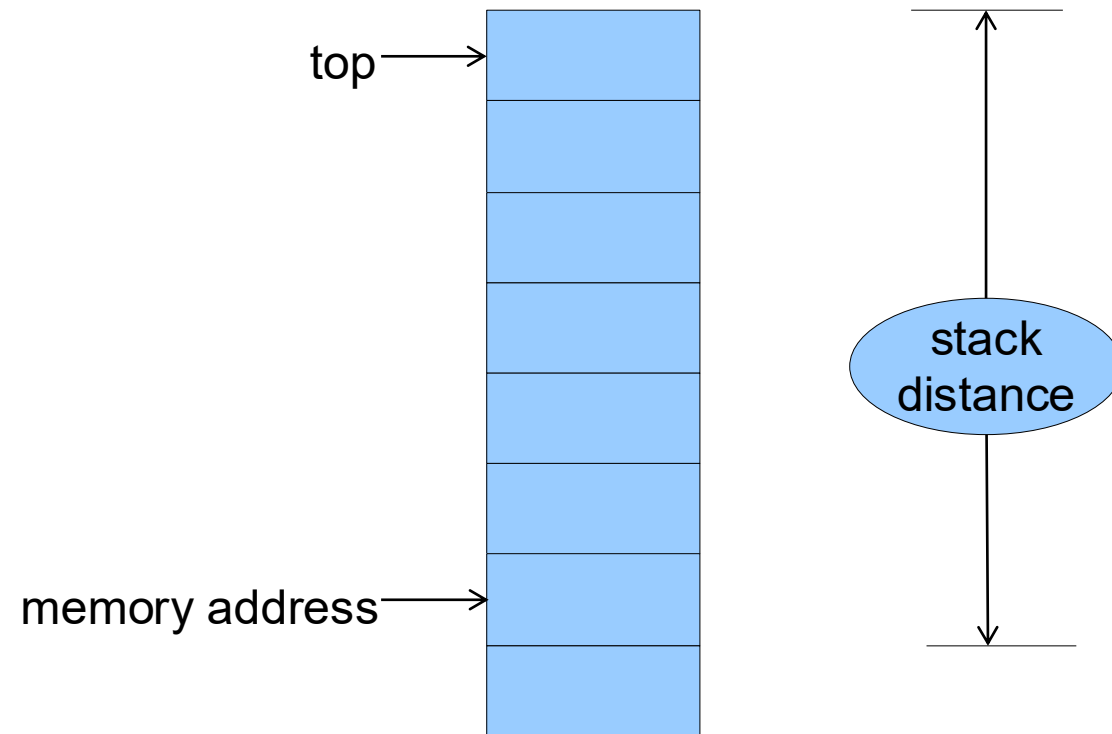
Spatial Locality

It is a concept that states that if a resource is accessed at some point of time, then most likely similar resources will be accessed again in the near future.

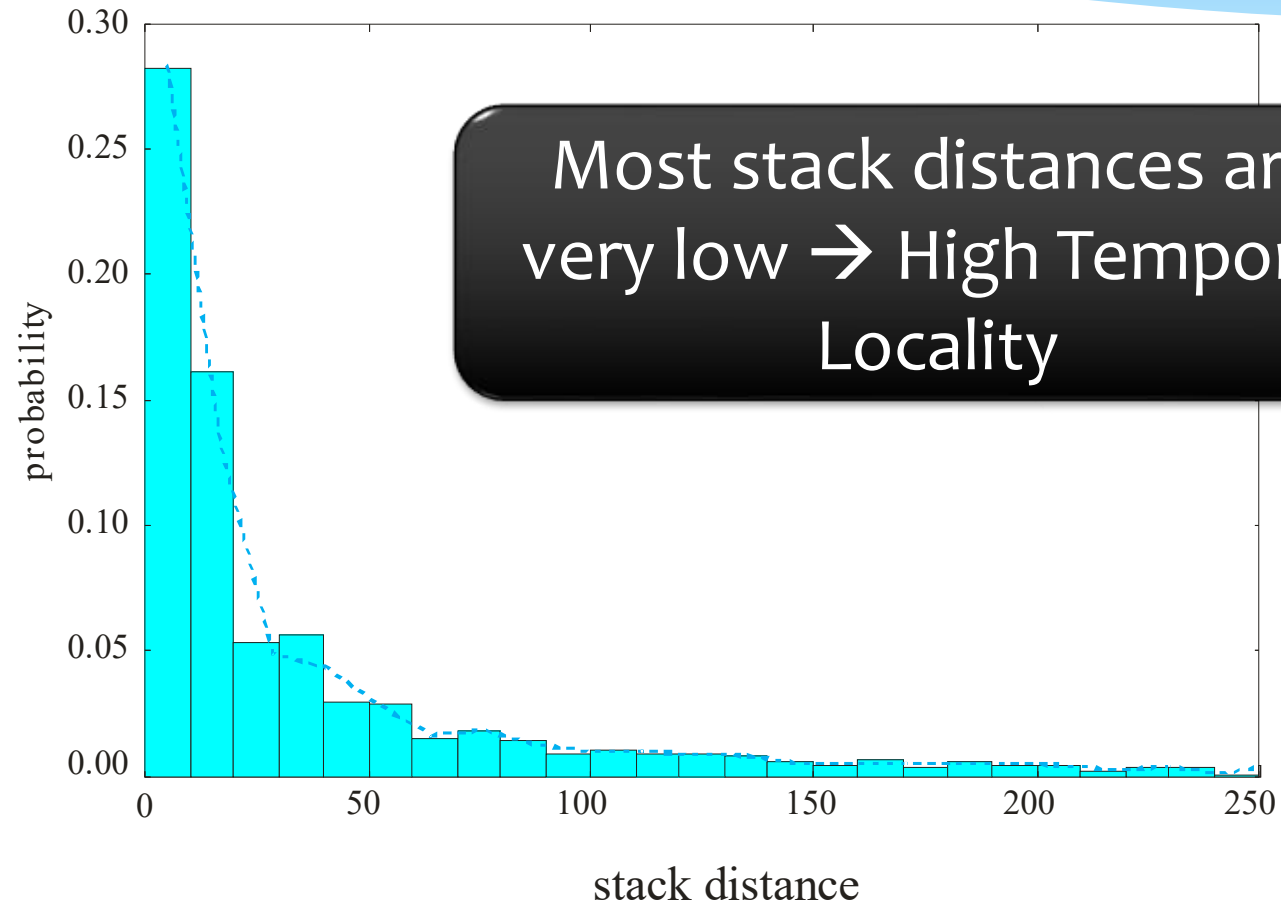
Temporal Locality in Programs

- * Let us verify if programs have temporal locality
- * Stack distance
 - * Have a **stack** to store memory addresses.
 - * Whenever, we **access an address** → we bring it to the top of the stack
 - * **Stack distance** → Distance from the top of the **stack** to where the element was found
 - * Quantifies reuse of addresses

Stack Distance



Stack Distance Distribution

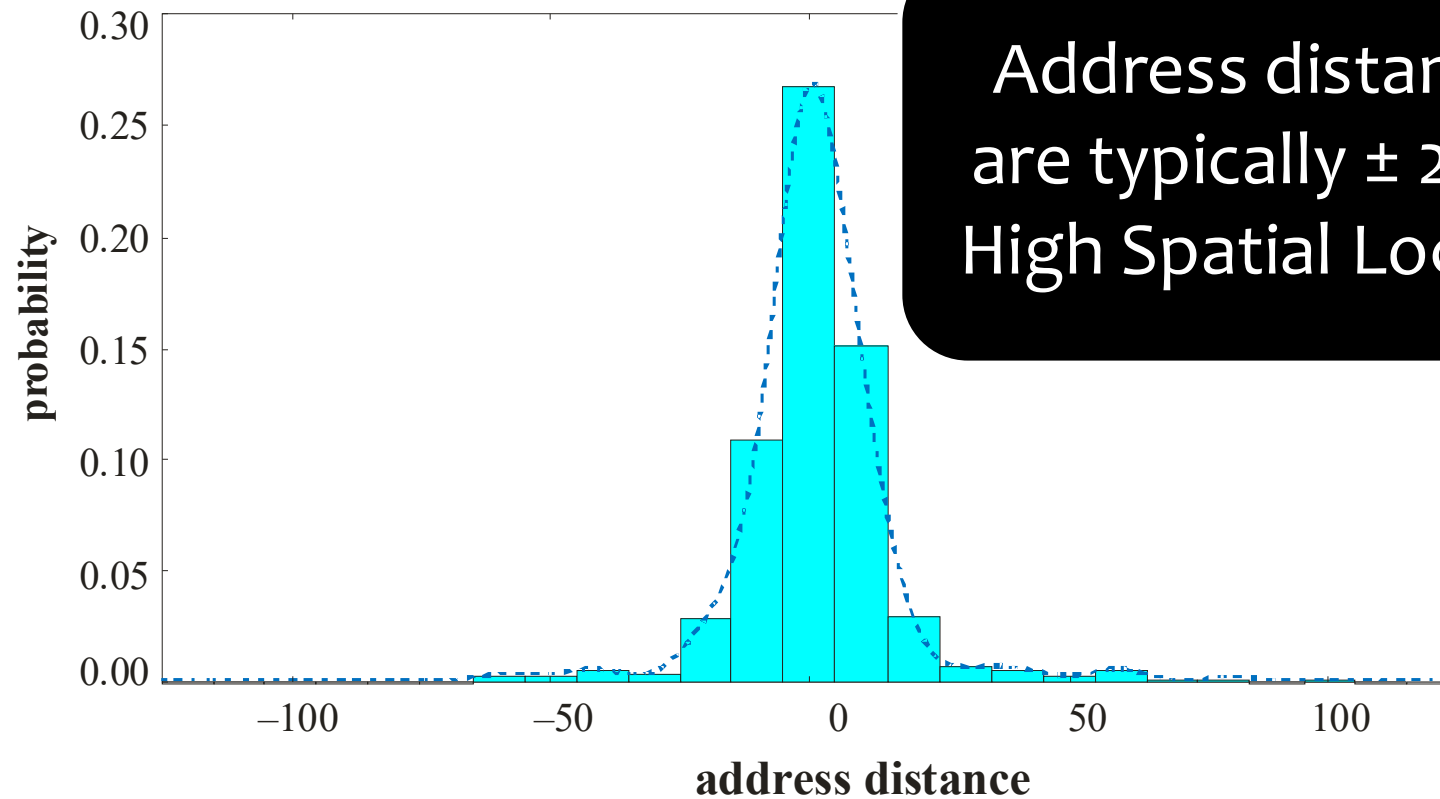


* Benchmark : Set of perl programs

Address Distance

- * Maintain a **sliding window** of the last K memory accesses
- * Address distance :
 - * The i^{th} **address distance** is the difference in the memory addresses of the i^{th} memory access, and the **closest** address in the set of last K memory accesses.
 - * Shows the **similarity** in addresses

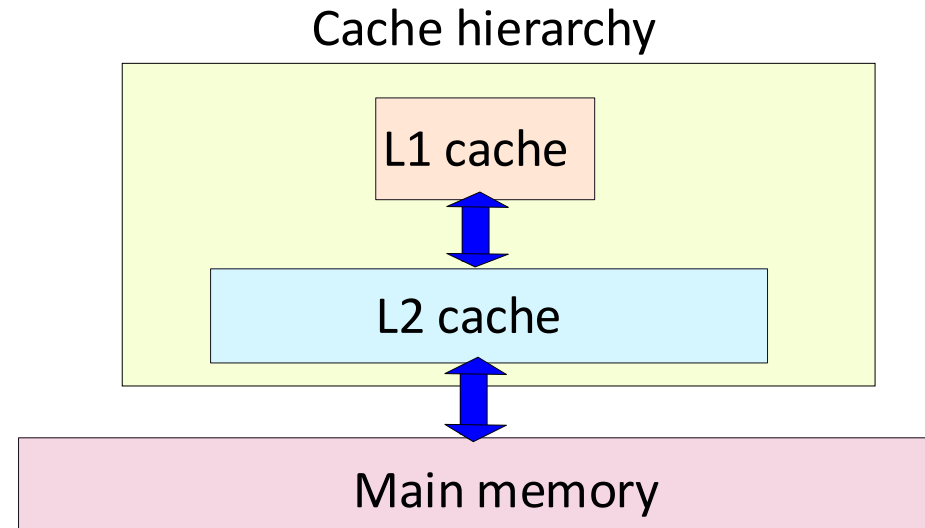
Address Distance Distribution



Address distances
are typically $\pm 20 \rightarrow$
High Spatial Locality

K=10, benchmark consisting of *perl* programs

Exploiting Temporal Locality



- * Use a **hierarchical memory** system
- * L1 (SRAM cells), L2 (SRAM cells), Main Memory (DRAM cells)